Day 6 - Transformers and Transfer Learning

Attention instead of memory - and how to reuse a pre-trained network

Annika Stiehl · Deep Learning · M.Sc. · SRH

Today

Time	
09:00 - 10:45	Attention: every word looks at every other word • the Transformer - with videos and browser tools
11:00 - 12:00	Transformers for images and signals • **RNN or Transformer?**
13:00 - 14:15	Transfer learning: reuse a pre-trained network • Lab 6: teach YOLO to find brain tumours
14:30 - 16:00	Project time

💻 Open `day6a_attention` in Colab now - it downloads a small Transformer (1-2 min). For the lab after lunch: **T4 GPU** runtime.

Last time

- **RNN**: the same small layer reads the sequence step by step and keeps a memory h
- the gradient is multiplied at every step → **vanishing**: plain RNNs forget after 10-20 steps
- **LSTM / GRU**: gates protect the memory → long memories become learnable - but still **one memory vector, step by step**

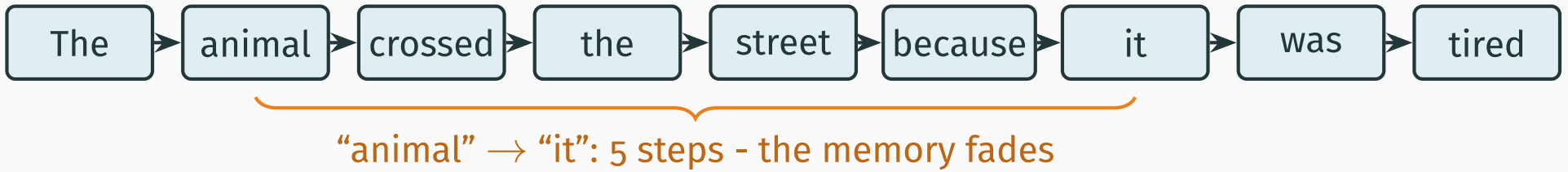
Today: what if every word could look at every other word **directly**?

Attention

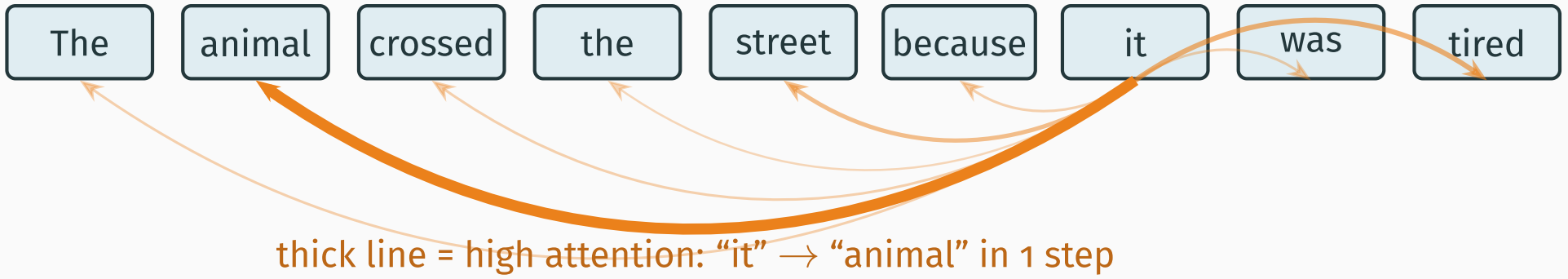
Every word looks at every other word

One memory is not enough

RNN: one memory, passed on step by step

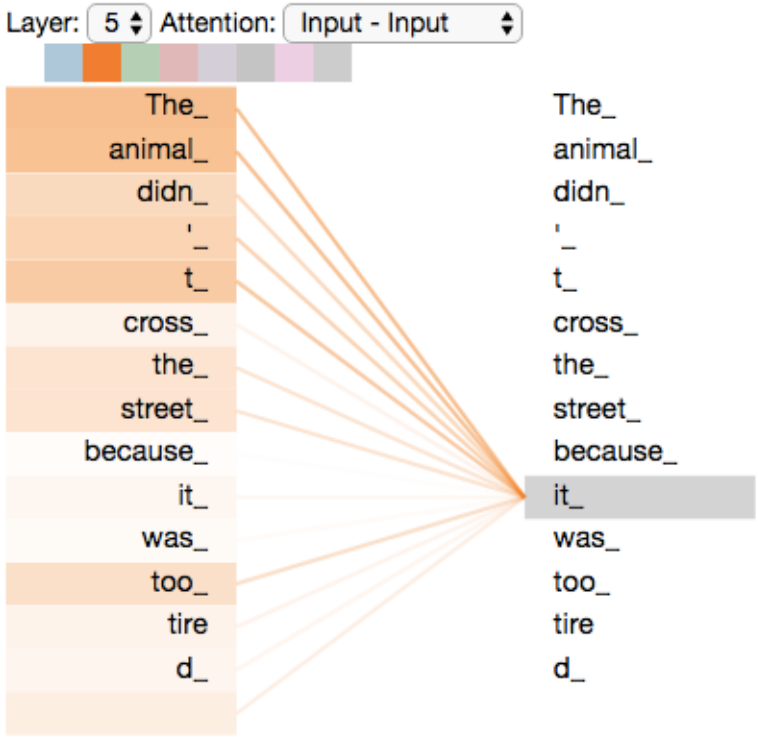


Attention: every word looks at every other word directly



- RNN: to connect "it" with "animal", the information must survive **5 steps** in one memory vector
- attention: "it" looks at **all** words at once and picks what matters - **1 step**, no matter how far

What is "it"?



"The animal didn't cross the street because **it** was too tired."

- for us: obvious - "it" = the animal
- change one word: "...because **it** was too **wide**" → "it" = the street
- the meaning of a word depends on **other words** - sometimes far away

The picture: a trained Transformer - the darker, the more "it" looks at that word.

Figure: J. Alammar, The Illustrated Transformer, CC BY-NC-SA 4.0

Video: what happens inside a Transformer?

3Blue1Brown: Transformers, the tech behind LLMs (Deep Learning, chapter 5)

youtube.com/watch?v=wjZofJX0v4M&t=183 - **3:03 → 6:36**
"Inside a transformer"

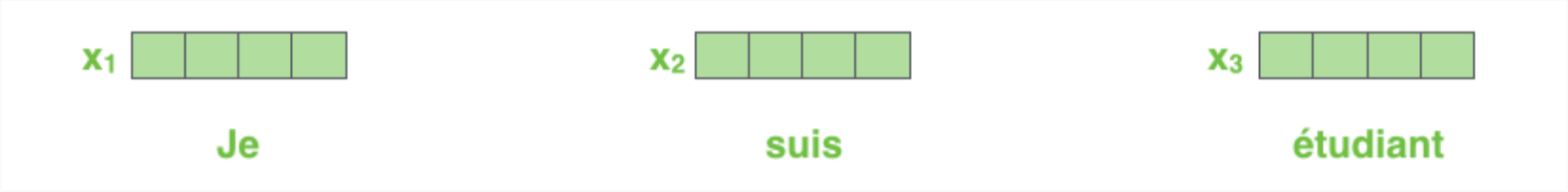
Watch for:

1. what is a **token**?
2. what happens in an **attention** block - and in an **MLP** block?
3. what comes out at the very end?

🎬 Watch (4 min)

Then: answer the 3 questions with your neighbour (2 min).

Words become vectors: embeddings



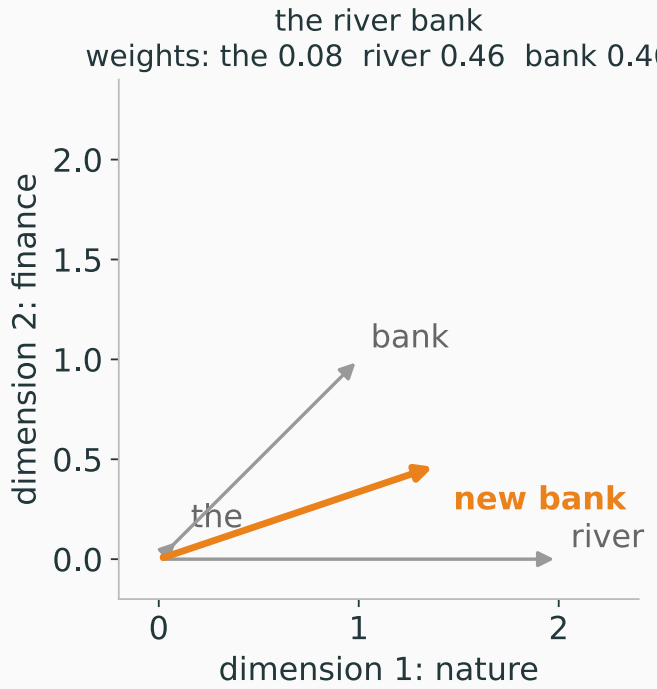
- every token gets a **vector** - e.g. 512 or 768 numbers (here: 4 boxes)
- learned: words with a similar meaning get **similar vectors**
- famous: king - man + woman ≈ queen

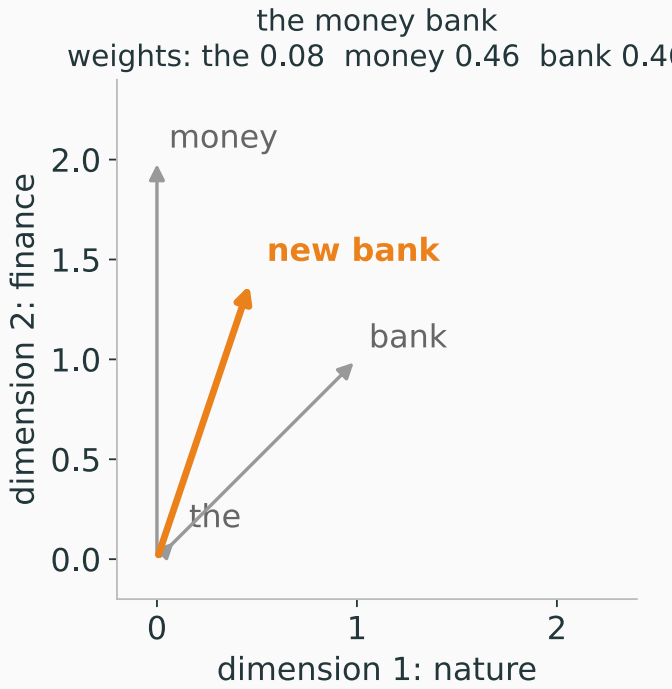
👥 Try it (5 min)

projector.tensorflow.org (Word2Vec 10K): search **heart**, then **brain**. Which 5 words are the closest neighbours?

Figure: J. Alammar, The Illustrated Transformer, CC BY-NC-SA 4.0

Attention by numbers: "river bank" or "money bank"?





1. "bank" compares itself with every word: **score** = dot product (day 2) → the(0.2), river(2.0), bank(2.0)
2. **softmax** turns the scores into weights between 0 and 1 that add up to 1 → 0.08, 0.46, 0.46
3. new "bank" = **weighted average** of all word vectors → it moves towards "river" - or towards "money"

Video: the attention pattern

3Blue1Brown: Attention in transformers, step-by-step (Deep Learning, chapter 6)

youtube.com/watch?v=eMlx5fFNoYc&t=99 - **1:39 → 11:08**
"Motivating examples", "The attention pattern", "Masking"

Watch for:

1. what is a **query**, what is a **key**?
2. what does the big **grid** (the attention pattern) show?
3. why does a model that writes text **mask** the future?

Query, key, value: attention is a search

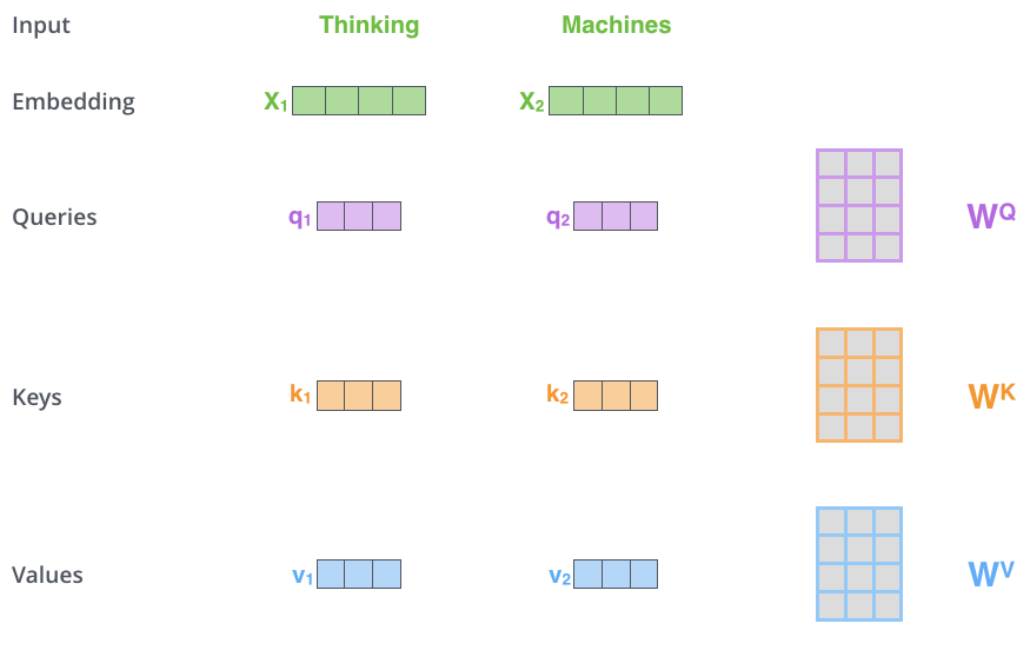


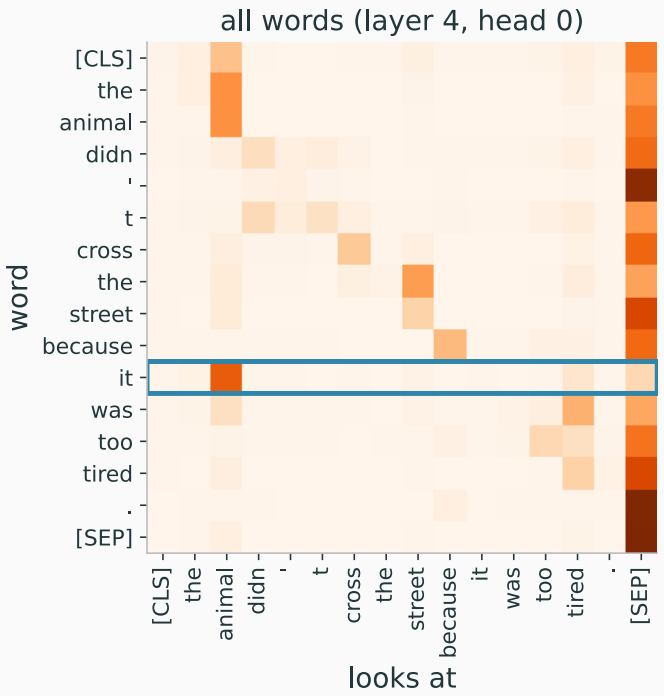
Figure: J. Alammar, The Illustrated Transformer, CC BY-NC-SA 4.0

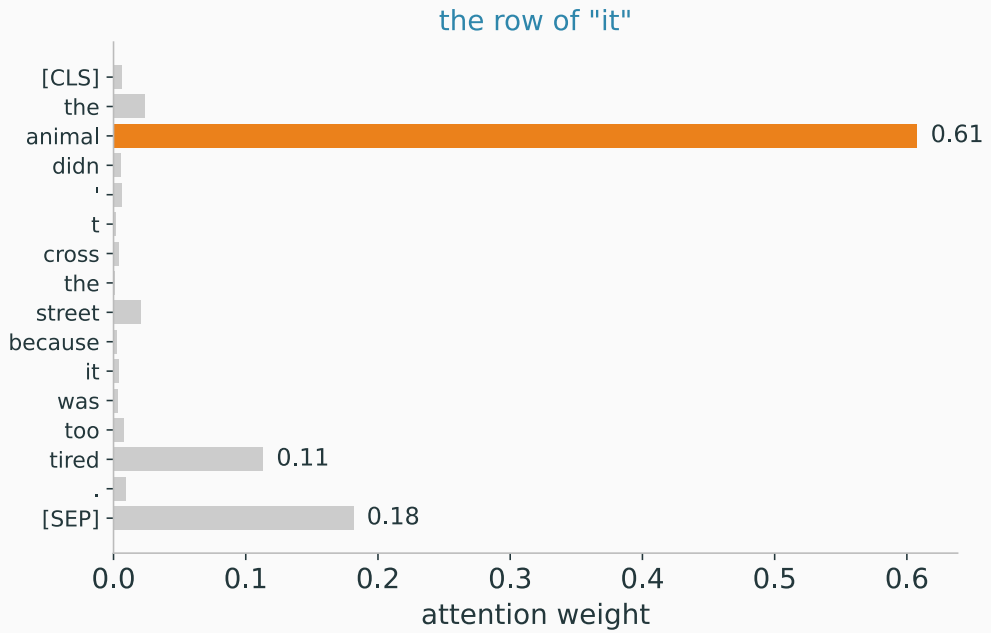
Like a library search:

	in the library	in attention
query	what you type in	what this word looks for
key	the title of each book	what each word offers
value	the content of the book	what each word passes on

- every word makes its q, k, v with **3 learned weight matrices**
- weight = how well query and key match (dot product → softmax)

Attention in a real model: where does "it" look?





DistilBERT (6 layers × 12 heads, pre-trained on Wikipedia and books) - layer 4, head 0: "it" puts **0.61** of its attention on **"animal"**. Nobody told the model.

Many heads look at different things

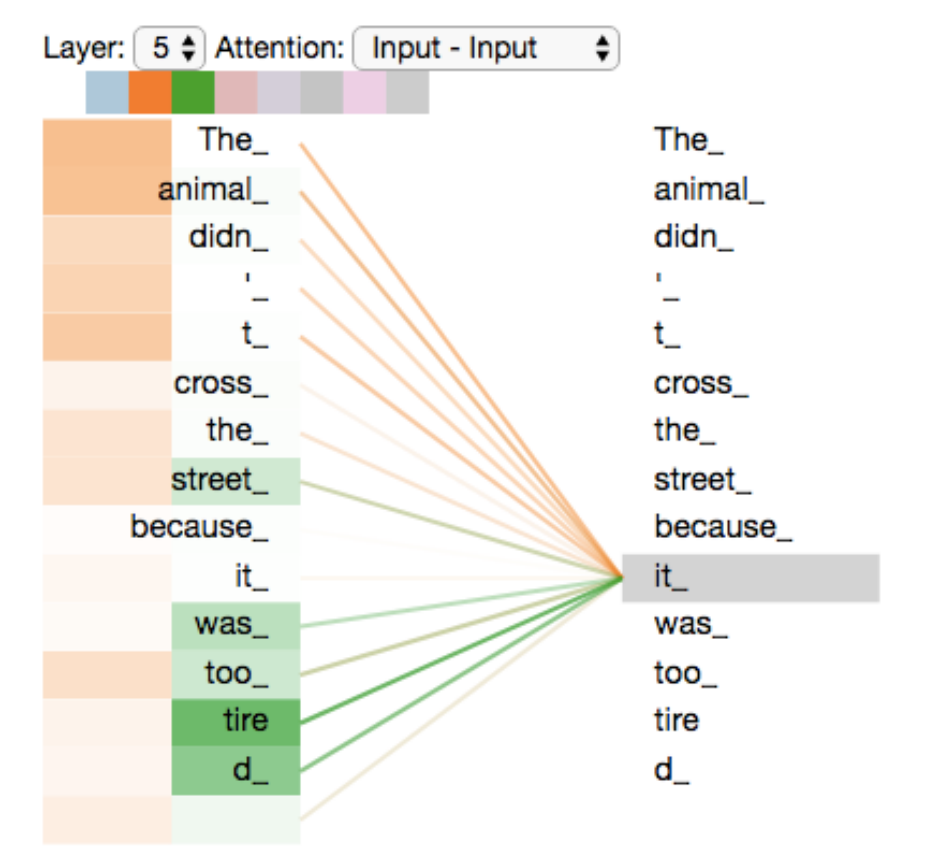
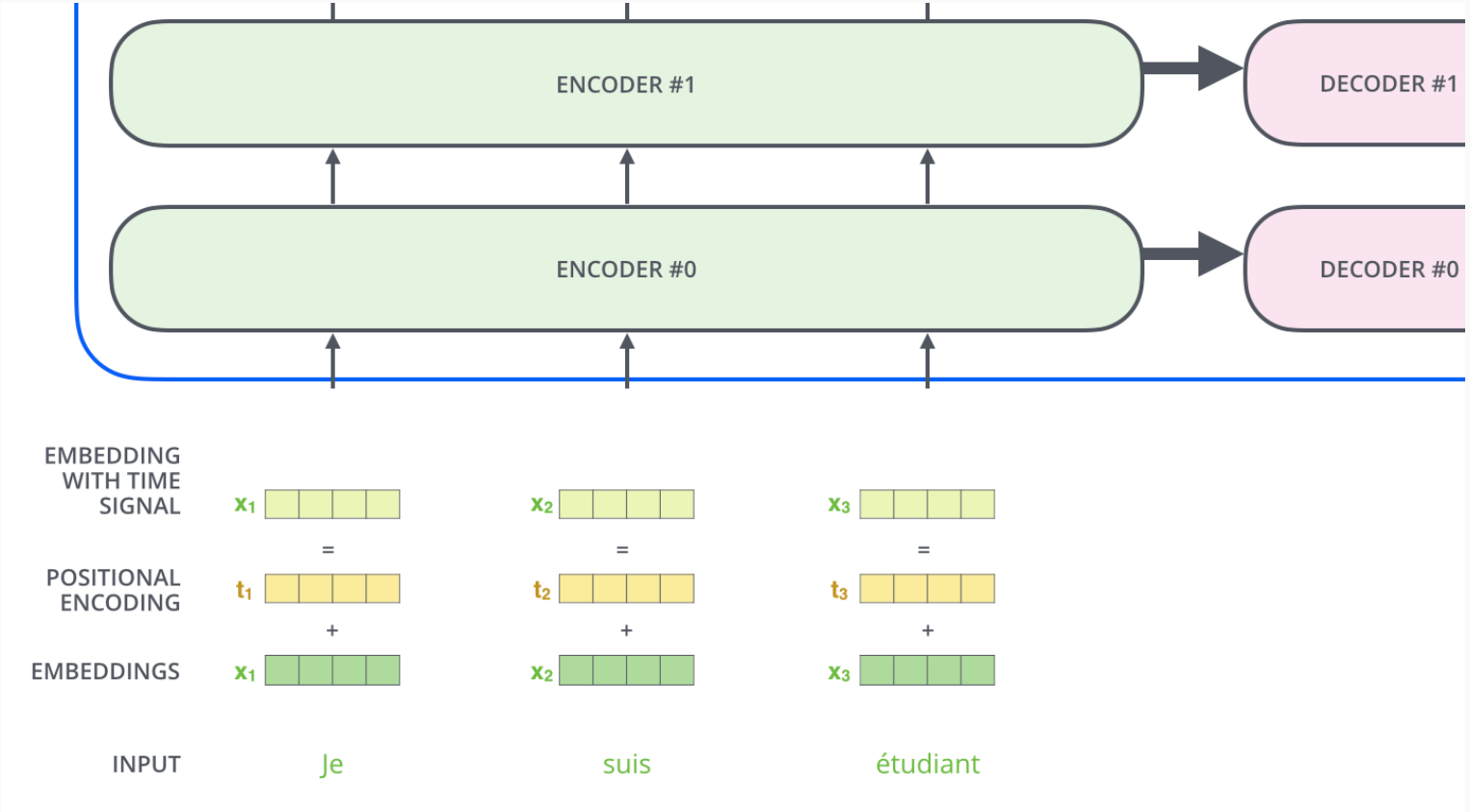


Figure: J. Alammar, The Illustrated Transformer, CC BY-NC-SA 4.0

- one attention = one way to look → **multi-head attention**: 8-12 heads in parallel, each with its own q, k, v
- here: one head links "it" to "the animal", another to "tired"
- the results of all heads are put together and mixed by one more layer

🎬 3Blue1Brown chapter 6, **19:19 → 22:16**
"Multiple heads" (3 min)

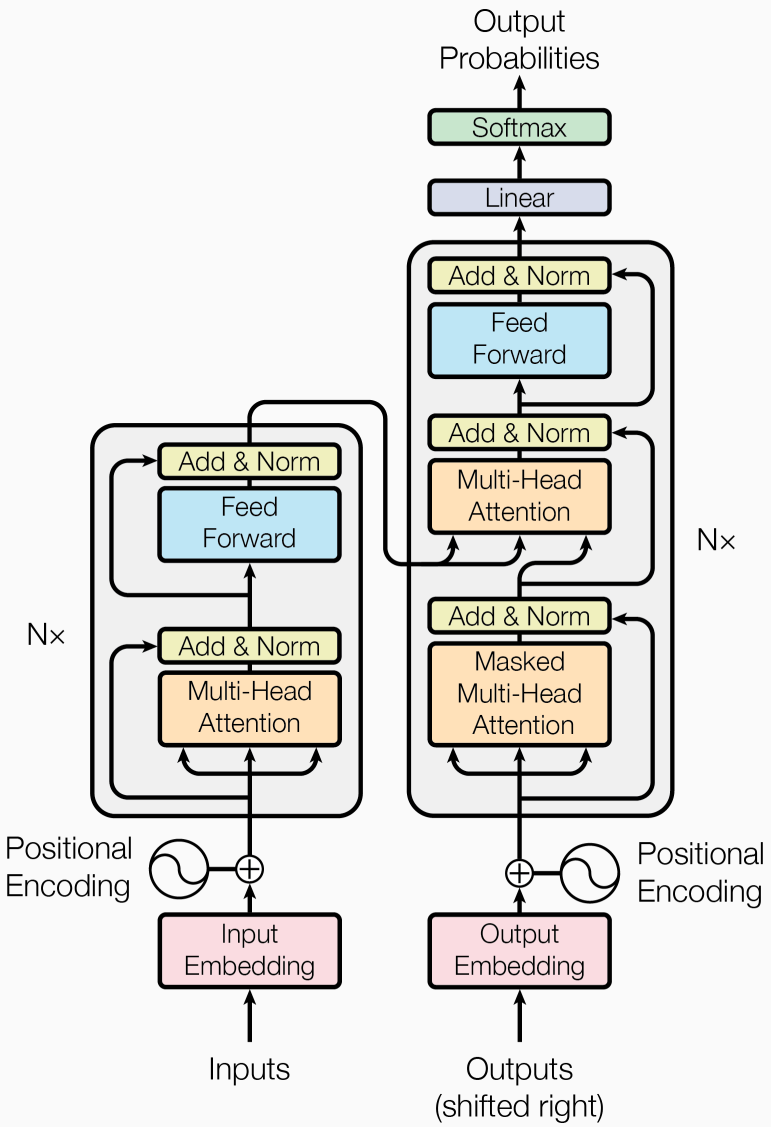
Word order: positional encoding



- attention compares every word with every word - it does **not know the order**: "dog bites man" = "man bites dog"
- fix: add a **position vector** to every word vector - position 1, 2, 3 ... get different patterns
- the RNN knew the order for free (step by step)

Figure: J. Alammar, The Illustrated Transformer, CC BY-NC-SA 4.0

The Transformer block



One block (× N, e.g. 6-96 times):

1. **multi-head attention** - words exchange information
2. **feed forward** = a small MLP (day 2) for every word on its own
3. **add & norm** = shortcut (ResNet, day 4) + layer normalisation

Two halves:

- left **encoder**: reads the whole input → **BERT** (understanding text)
- right **decoder**: writes word by word, masked → **GPT** (ChatGPT)

Figure 1 of Vaswani et al., Attention Is All You Need, NeurIPS 2017

Try it: Transformer Explainer

poloclub.github.io/transformer-explainer - a real GPT-2 in your browser

1. type a sentence, e.g. "The patient was taken to the" → **Generate**: which next word?
2. click into the **attention** part: which words does the last word look at?
3. move the **temperature** slider to 0.2 and to 2 - what happens to the probabilities?

👥 Your turn (10 min)

In pairs, in your browser. Find one thing that surprises you - we collect 2-3 at the end.

Look at it: a whole GPT in 3D

bbycroft.net/llm - LLM Visualization (B. Bycroft)

- every number of a small GPT as a little block - you can follow one token through all layers
- choose **nano-gpt** (85 000 weights), then compare with **GPT-3** (175 000 000 000)

🔑 Same recipe, bigger

ChatGPT = the same blocks as on the last slides - many more of them, trained on much more text.

Pre-trained Transformers: learn first, use later

- **pre-training**: hide words in billions of sentences and let the model guess them (BERT) - or let it guess the next word (GPT) → **no labels needed**
- the model learns grammar, facts, medicine … from the text alone
- then: **adapt** it to your task with a little labelled data → **transfer learning** (after lunch)

❓ Think about it

DistilBERT reads "The patient had an epileptic [MASK] during the night." - which word will it guess, and how sure is it?

Cheat sheet: the Transformer in one slide

token + embedding	a piece of text → a learned vector
position	a position vector is added - attention alone does not know the order
attention	every token becomes a weighted mix of all tokens; weights = how well query and key match (softmax)
multi-head	several attentions in parallel, each looks for something else
block	attention → MLP, each with shortcut + normalisation; N blocks stacked
encoder / decoder	encoder reads everything (BERT); decoder writes word by word, future masked (GPT)
pre-training	guess hidden or next words in huge amounts of text - no labels needed

RNN or Transformer?

Transformers everywhere - and when the RNN is still the better choice

Images as sequences: the Vision Transformer



- cut the image into **patches** (e.g. 16 × 16 pixels) - every patch is one token, like a word
- add a position vector → the same Transformer as for text → class (ViT, Google 2020)
- no convolution at all - but it needs **lots of data** or pre-training (ImageNet-21k: 14 million images)

Signals as sequences

- **ECG / EEG**: cut the signal into short pieces (e.g. 1 s) → every piece is one token
- many channels: one token per channel and piece
- **foundation models** for EEG and ECG: pre-trained on thousands of hours of recordings, then fine-tuned for one task (e.g. sleep stages, seizures)
- same idea as ViT: **patches → tokens → Transformer**

❓ Think about it

Our heartbeats have 140 samples. With pieces of 10 samples - how many tokens is one beat?

The price: every token with every token

sequence	tokens	pairs for attention
one sentence	20	400
one heartbeat (140 samples)	140	19 600
a 10-page report	5 000	25 000 000
1 hour of ECG, 500 samples per second	1 800 000	3 240 000 000 000

- attention: **n^2 pairs** - double the length → 4 × the work and memory
- an RNN: **n steps** - double the length → 2 × the work, and the memory stays the same size
- → long signals: patches, windows, or an RNN / CNN

RNN vs Transformer: properties

	RNN (LSTM / GRU)	Transformer
reads	step by step	all tokens at once
long-range	through one memory - fades	direct connection, any distance
work per sequence	grows with n	grows with $\mathbf{n^2}$
training on a GPU	slow - steps must wait for each other	fast - everything in parallel
running live (one new sample at a time)	cheap: update one memory	the history grows - more work per new token
word order	built in	must be added (positions)
data needed	works with little data	needs a lot - or a pre-trained model
pre-trained models	few	many (text, images, ECG, EEG ...)
look inside	memory is hard to read	attention maps (a hint, no proof)

What does each one need?

RNN / LSTM / GRU fits when ...

- the data is **small** (hundreds to thousands of examples)
- the model runs **live** on a small device (watch, sensor)
- sequences are **very long** and arrive sample by sample
- there is no GPU for training

Transformer fits when ...

- a **pre-trained model** exists for your kind of data
- there is **a lot of data** (or you fine-tune)
- **long-range** relations matter (text, whole documents)
- a **GPU** is available - and the sequence is not too long (or is cut into patches)

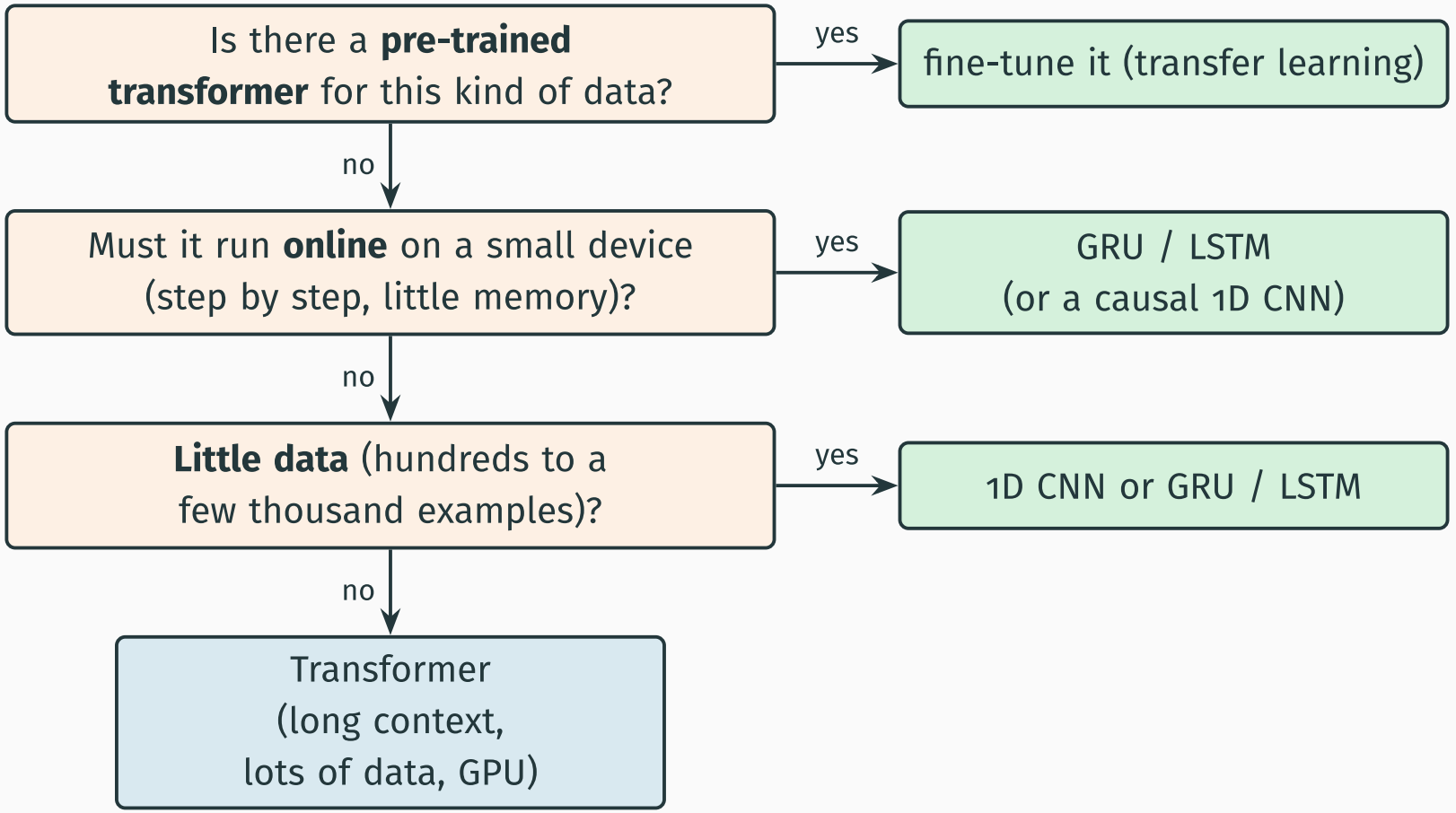
Your turn: RNN, CNN or Transformer?

1. a **smartwatch** that warns of an epileptic seizure while the patient sleeps
2. a **chatbot** that answers questions about the course
3. **500** labelled 10-s ECGs - normal or atrial fibrillation
4. **sleep stages** from 10 000 nights of EEG, a hospital GPU cluster
5. **translate** a doctor's letter from German to English

👥 In pairs (8 min)

Choose one model type for each case - and write down **one reason**. We collect the answers.

A decision guide



Always: try a simple **baseline** first (day 3) - and compare on the same split.

Outlook: RNNs are back?

- the n^2 problem of attention is the motivation for new **recurrent** models that train in parallel like a Transformer:
- **xLSTM** (Beck, ..., Hochreiter, 2024) - an LSTM with new gates and memory, from the inventor of the LSTM
- **Mamba** (Gu & Dao, 2023) - a "state space model": grows with n, not n^2
- current research: mix both - attention for some layers, recurrence for others

Transfer learning

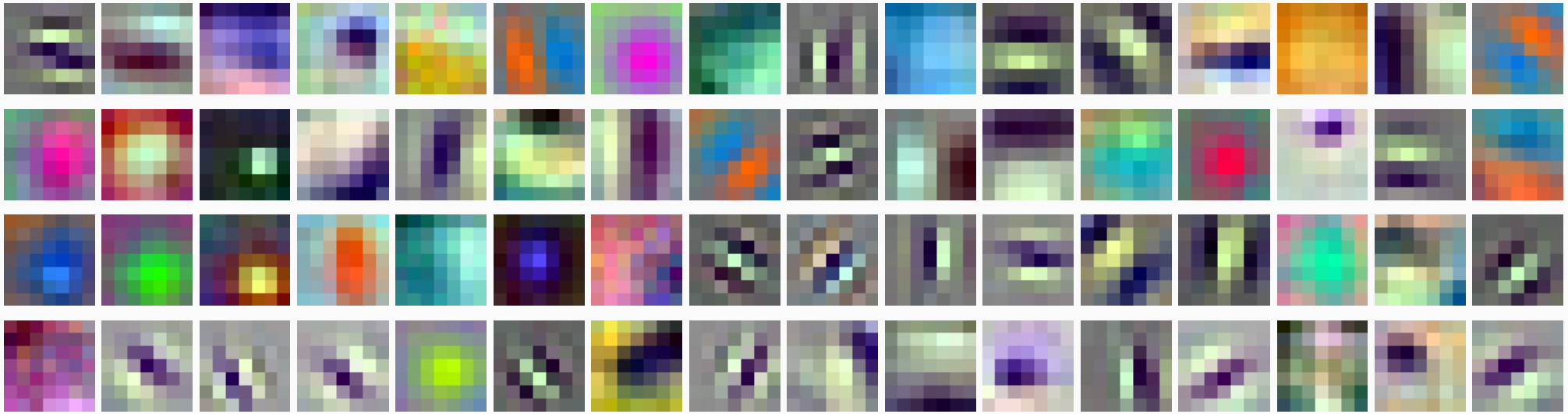
Reuse a pre-trained network - 13:00-14:15 · Lab 6: YOLO finds brain tumours

Labels are expensive - pre-trained networks are free

dataset	images	labelled by
ImageNet	1 200 000 (1 000 classes)	crowd workers
COCO	118 000 (80 classes, boxes)	crowd workers
our brain scans	893	doctors
your challenge (DermaMNIST)	7 007	dermatologists

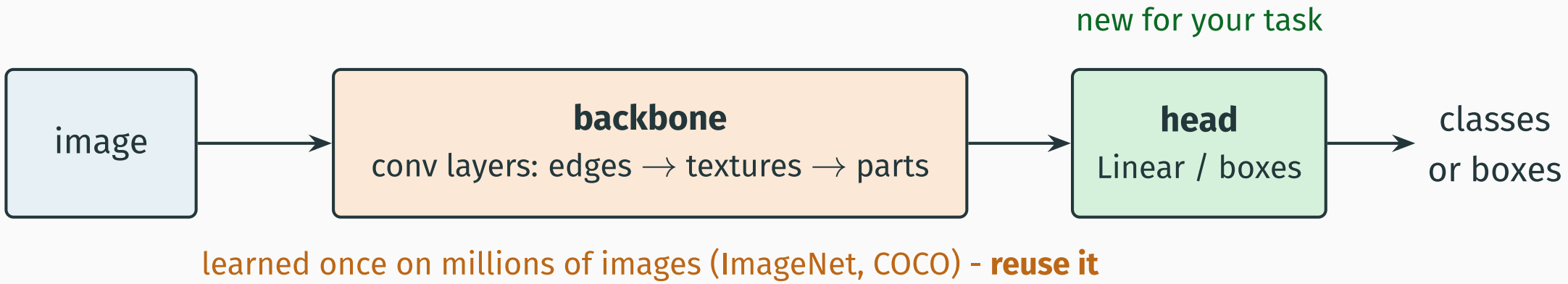
- medical labels need experts → small datasets
- but the **first layers** of an image network learn the same things for every task

What does a pre-trained network know?



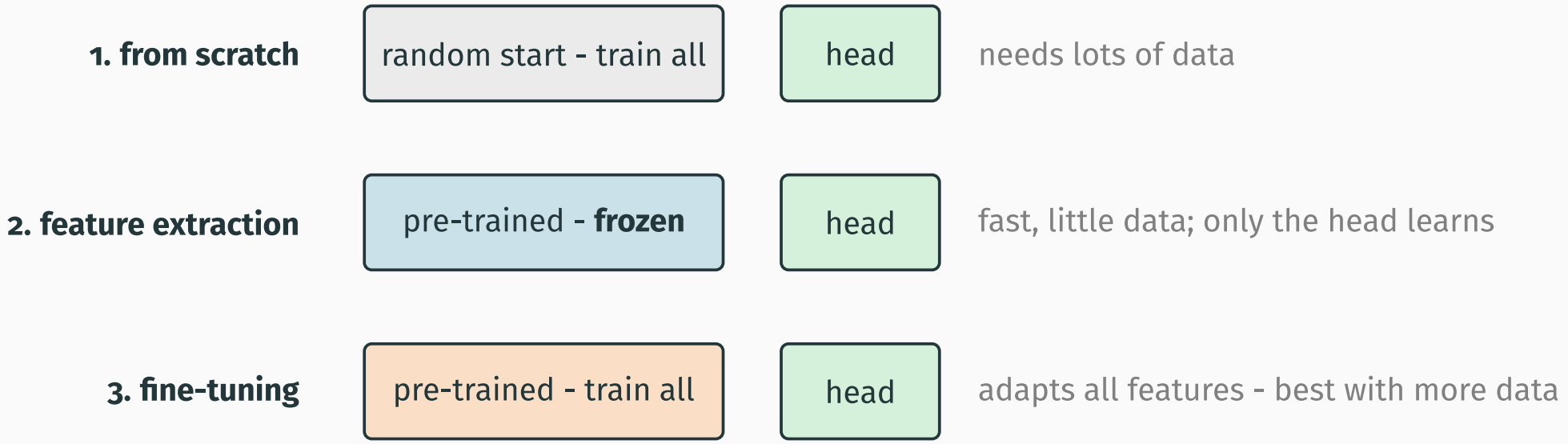
- the 64 filters of the **first layer** of ResNet-18, trained on ImageNet: **edges** in all directions, **colour blobs**
- deeper layers: textures → patterns → parts → objects (day 4)
- edges, textures and round shapes are also useful in an MRI, an X-ray or a skin photo

Backbone + head



- **backbone**: all layers that turn the image into features - pre-trained, reused
- **head**: the last layer(s) for your classes or boxes - always new
- ImageNet: 1 000 classes → your challenge: 7 classes → `model.fc = nn.Linear(512, 7)`

Three strategies



- **feature extraction**: `freeze` the backbone (`requires_grad = False`) - only the head learns
- **fine-tuning**: start from the pre-trained weights, train everything with a **small learning rate** (e.g. 10 × smaller)
- in the lab you try all three - with one word of code each

Which strategy when?

	your data is similar to the pre-training data	**your data is different (e.g. MRI vs photos)**
little data (≤ a few hundred)	feature extraction	feature extraction - maybe also fine-tune the last blocks
more data (thousands)	fine-tuning	fine-tuning (all layers)

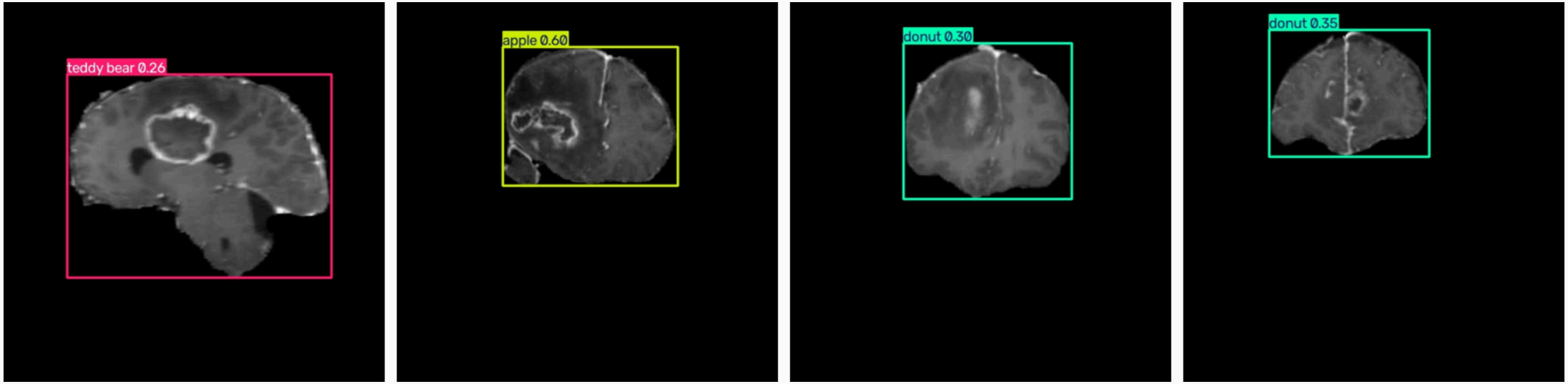
- "from scratch" only with **a lot of data** - or when no pre-trained model fits at all
- the input must look like the pre-training: image size, **normalisation** (ImageNet mean / std)

Where to get pre-trained models

library	what	one line
torchvision	ResNet, EfficientNet, ViT … (ImageNet)	`resnet18(weights="DEFAULT")`
timm	> 1 000 image models	`timm.create_model("resnet18", pretrained=True, num_classes=7)`
Hugging Face	text, images, audio, medicine	`AutoModel.from_pretrained("distilbert-base-uncased")`
Ultralytics	YOLO for detection	`YOLO("yolo11n.pt")`

Always check the **licence** and the **model card**: what data, which task, which limits?

A COCO detector looks at brain scans …



YOLO pre-trained on COCO (80 everyday classes): it puts a box around the **whole brain** and calls it **donut**, **apple** or **teddy bear** - it can only answer with the classes it knows. But its backbone finds edges and round shapes.

Lab 6: teach YOLO to find brain tumours

💻 `day6_yolo.ipynb` in Colab · **T4 GPU** · 893 training images · a few lines of code

Part 1 - the COCO model as it is (~5 min)
Part 2 - the data (~5 min): images and boxes
Part 3 - fine-tune YOLO (~15 min): one line - then: donuts or tumours?

Part 4 - is pre-training worth it? (~20 min)
from scratch vs frozen backbone vs fine-tuning - one word each

👥 How it works

In pairs. Run → change one word → compare in the table on the task sheet.

Project time

14:30-16:00

Project time

- **free work** in your teams on the challenge
- today's idea: a **pre-trained network** for your model
- questions? I come round to every team
- if you like: we look at a **paper** together

Dates

- **10 Nov**: send your model (`model.pt`)
- **11 Nov**: print the poster (at the latest) · poster presentations · leaderboard
- **15 Nov**: hand in on **CampusWeb**: link to the code, model, report (PDF), poster (PDF/PNG), app

Wrap-up

Summary

- **attention**: every token becomes a weighted mix of all tokens - query, key, value; many heads
- **Transformer**: attention + MLP blocks, positions added; encoder (BERT) or decoder (GPT); also for images and signals (patches)
- **RNN or Transformer**: live / small data / on-device → RNN or CNN; pre-trained model or lots of data + GPU → Transformer
- **transfer learning**: backbone + new head; feature extraction vs fine-tuning - almost always better than from scratch with little data

📄 Everything on one page: Stanford CME 295 cheat sheet: Transformers & large language models (PDF)

Lab 6 – Transfer learning: YOLO finds brain tumours

Notebook: `day6_yolo.ipynb` (course repository) · open it in Colab · **Runtime** → **T4 GPU** · work in **pairs** · what you do not finish in class is **homework**.

How this lab works

Today you write almost no code. **Run** the cells, **change one word** where it says so, **look** at the pictures and **write the numbers** into the table on this sheet. First run the two **setup cells** (they install Ultralytics and download 1 116 brain scans, about 1 minute).

Data: 893 training and 223 validation brain scans (MRI/CT, 512×512). Every image has a box around a region: **positive** (tumour) or **negative** (no tumour). **Model:** YOLO11n (2.6 million weights), pre-trained on COCO (118 000 photos, 80 everyday classes).

Part 1 – The COCO model as it is — about 5 min

☐ 1 What does a COCO detector see in a brain scan? — 5 min

Run the cell: the COCO model looks at 8 validation images.
Question: which classes does it find – and why exactly these?

Part 2 – The data — about 5 min

☐ 2 Images and boxes — 5 min

8 training images with the boxes from the labels (red = positive, blue = negative).
Question: how big are the boxes? Could *you* tell positive from negative?

Part 3 – Fine-tune YOLO — about 15 min

☐ 3 One line of code — 15 min

`YOLO("yolo11n.pt").train(data="brain-tumor.yaml", epochs=15, imgsz=320)`: start from the COCO weights, new head with 2 classes, train all layers (1–2 min on a T4).
mAP50 (mean average precision at IoU 0.5): 1 = every region found with a good box and the right class.
Question: what does the model find now? Which mistakes does it still make?

Part 4 – Is pre-training worth it? about 20 min

☐ 4 Three strategies – one word each 20 min

Copy the `train` line of Part 3 for the other two strategies and fill in the table.
Read the mAP50 after epoch 3 from the learning curves (plot at the end of Part 4).

strategy	start	code	mAP50 epoch 3	mAP50 epoch 15
from scratch	random	`YOLO("yolo11n.yaml")`		
feature extraction	COCO	`train(..., freeze=10)`		
fine-tuning	COCO	Part 3		

Question: what is the big difference in the first epochs – and what happens later?

☐ ★ Bonus More epochs, bigger model, your own data homework

(1) `EPOCHS = 50`. (2) Less data: `fraction=0.2` – how big is the gap now? (3) `YOLO("yolo11s.pt")`. (4) A dataset from `universe.roboflow.com` in YOLO format – same line of code.